

Flush+Monitor sui kernel Linux moderni

Riproduzione di un page-cache side channel, adattamento della primitiva di Monitor, misura della risoluzione temporale e valutazione di una mitigazione constant-access.

Progetto d'esame basato su Eviction Notice (NDSS 2026)

Domanda centrale: la residency di una pagina condivisa può ancora essere usata come oracolo affidabile su Linux 7.2, e con quale finestra minima di osservazione?

Domanda di ricerca e percorso della presentazione

Domanda. Dopo le mitigazioni di mincore e cachestat e dopo il cambiamento della semantica di RWF_NOWAIT, è ancora possibile costruire un ciclo Flush+Monitor ripetibile tra utenti diversi?

- | | |
|-------------------------------|--|
| 1 Fondamenti | Page cache, residency e costruzione del canale laterale. |
| 2 Threat model e paper | Attori, assunzioni, mitigazioni storiche e risultati di Eviction Notice. |
| 3 Adattamento moderno | Differenza fra 5.15 e 7.2; readahead asincrono e ripristino dello stato. |
| 4 Valutazione | Protocollo, metriche, baseline e sweep della finestra di osservazione. |
| 5 Mitigazione e demo | Constant-access, costo, cross-container e UI redressing controllato. |

Il confronto separa tre livelli: riproduzione del paper, cambiamento della semantica del kernel ed estensione sperimentale sviluppata nel progetto.

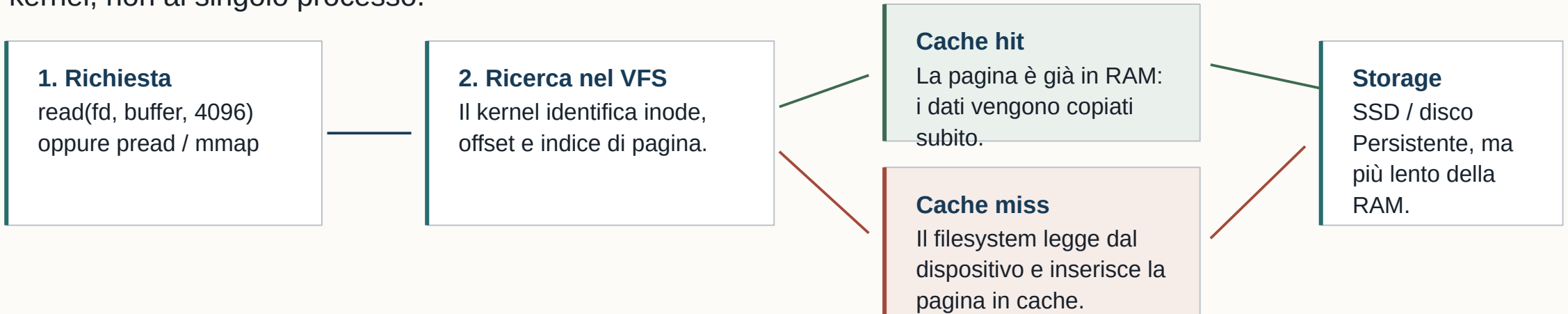
SEZIONE 1

Dalla page cache al canale laterale

Prima di parlare di attacco occorre capire quale stato mantiene il kernel, perché tale stato è condiviso e in quale modo una singola pagina può codificare un'informazione sull'attività di un altro processo.

Che cosa fa la page cache

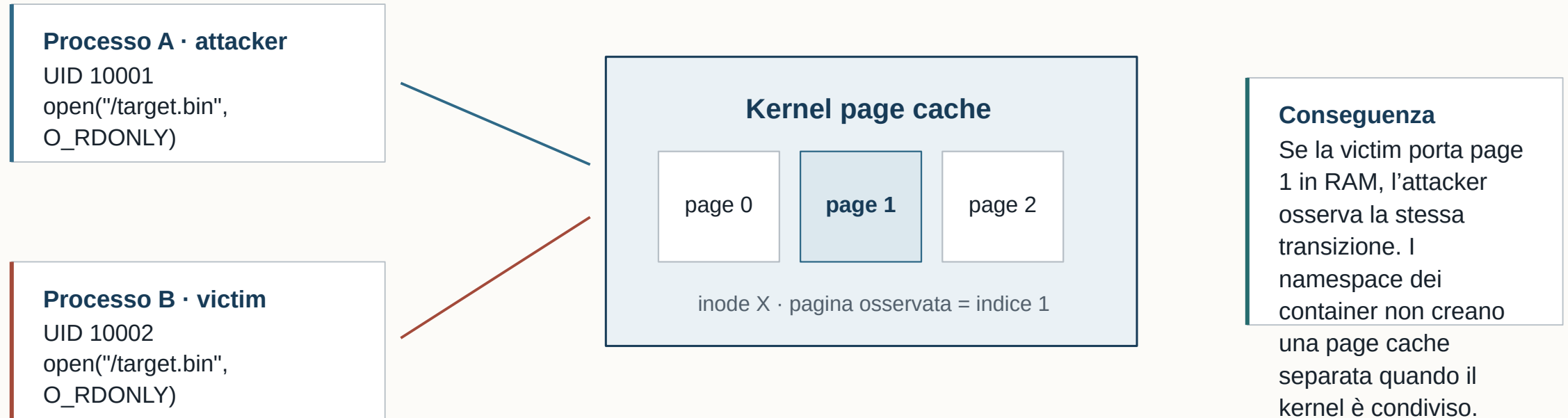
La page cache è la copia in RAM dei dati letti dai file. Riduce gli accessi al dispositivo di memoria e appartiene al kernel, non al singolo processo.



- Nel nostro sistema una pagina è di 4096 byte: questa è anche la risoluzione spaziale naturale dell'esperimento.
- La page cache non va confusa con le cache L1/L2/L3 della CPU: qui osserviamo uno stato software globale gestito dal kernel.
- Il contenuto della pagina non è il segreto osservato. Il segreto è la sua presenza o assenza in RAM in un determinato istante.

La pagina è identificata dal file, non dal processo

Due processi che aprono lo stesso file non ricevono due copie indipendenti della cache. Le loro letture convergono sull'address_space associato allo stesso inode.



- Percorsi diversi possono riferirsi allo stesso inode; viceversa, due copie distinte del file non condividono necessariamente la stessa pagina.
- Memoria anonima e pagine private della victim non rientrano in questo canale: serve un oggetto file-backed condiviso.

Residency: il bit osservabile

Definiamo $C(f,p,t) \in \{0,1\}$. La variabile non descrive il contenuto della pagina, ma soltanto se il kernel la mantiene nella page cache all'istante t .

$$C(f,p,t) = 0$$

Pagina assente. Una lettura richiederebbe I/O o readahead.

$$C(f,p,t) = 1$$

Pagina residente. Una lettura può essere soddisfatta dalla RAM.

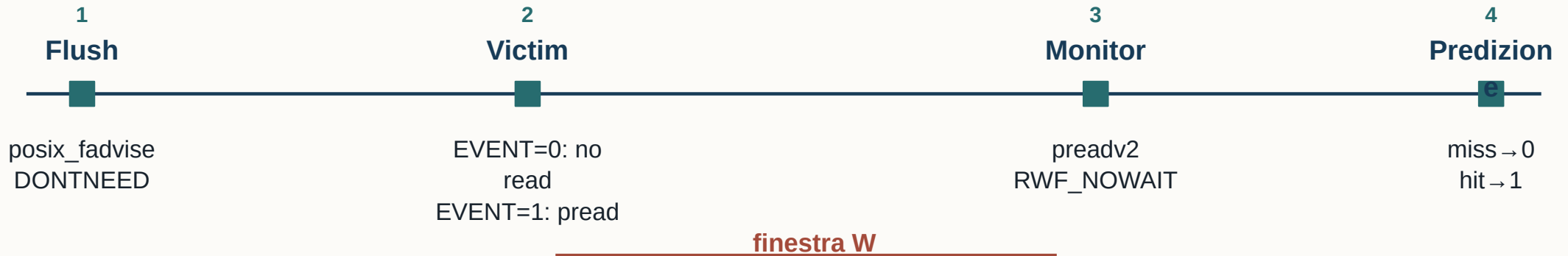
Transizioni rilevanti

Operazione	Stato prima	Stato dopo	Interpretazione
Flush della pagina	0 oppure 1	0	L'attacker imposta un baseline noto.
Lettura della victim	0	1	L'attività della victim modifica metadato condiviso.
Monitor	0 oppure 1	osservato	L'attacker classifica lo stato senza leggere dati privati.

L'informazione nasce dalla correlazione: se una specifica attività è l'unica che può produrre la transizione $0 \rightarrow 1$ durante la finestra, la residency diventa un indicatore di quell'attività.

Un round di Flush+Monitor, passo per passo

L'attacker forza prima $C=0$, lascia alla victim un intervallo W e infine misura se la pagina è tornata residente.



Caso **EVENT=0**

La victim non legge la pagina.
 C resta 0 → il Monitor restituisce `EAGAIN` → predizione 0.

Caso **EVENT=1**

La victim legge la pagina.
 C passa a 1 → il Monitor legge 1 byte → predizione 1.

La risoluzione temporale migliora quando W diminuisce; sotto una certa soglia la lettura della victim non termina prima del Monitor e compaiono falsi negativi.

Le quattro primitive e le cinque tecniche del paper

Il paper separa le operazioni elementari dalle tecniche d’attacco ottenute combinandole.

Primitiva	Funzione	Implementazione / segnale	Osservazione
Flush	Rimuove una pagina nota	POSIX_FADV_DONTNEED	Baseline deterministico e non privilegiato.
Evict	Provoca rimozione indiretta	Pressione sulla memoria	Meno deterministico, utile senza Flush diretto.
Monitor	Interroga la residency	cachestat o preadv2 NOWAIT	Idealmente non porta in cache una pagina assente.
Reload	Legge e misura il tempo	pread / mmap + timing	La misura stessa carica la pagina.

Combinazioni studiate

Flush+Monitor

Flush+Reload

Flush+Flush

Evict+Monitor

Evict+Reload

Questo progetto si concentra su Flush+Monitor: il Flush rende noto lo stato iniziale; il Monitor prova a osservare la residency senza confondere una propria lettura con l’attività della victim.

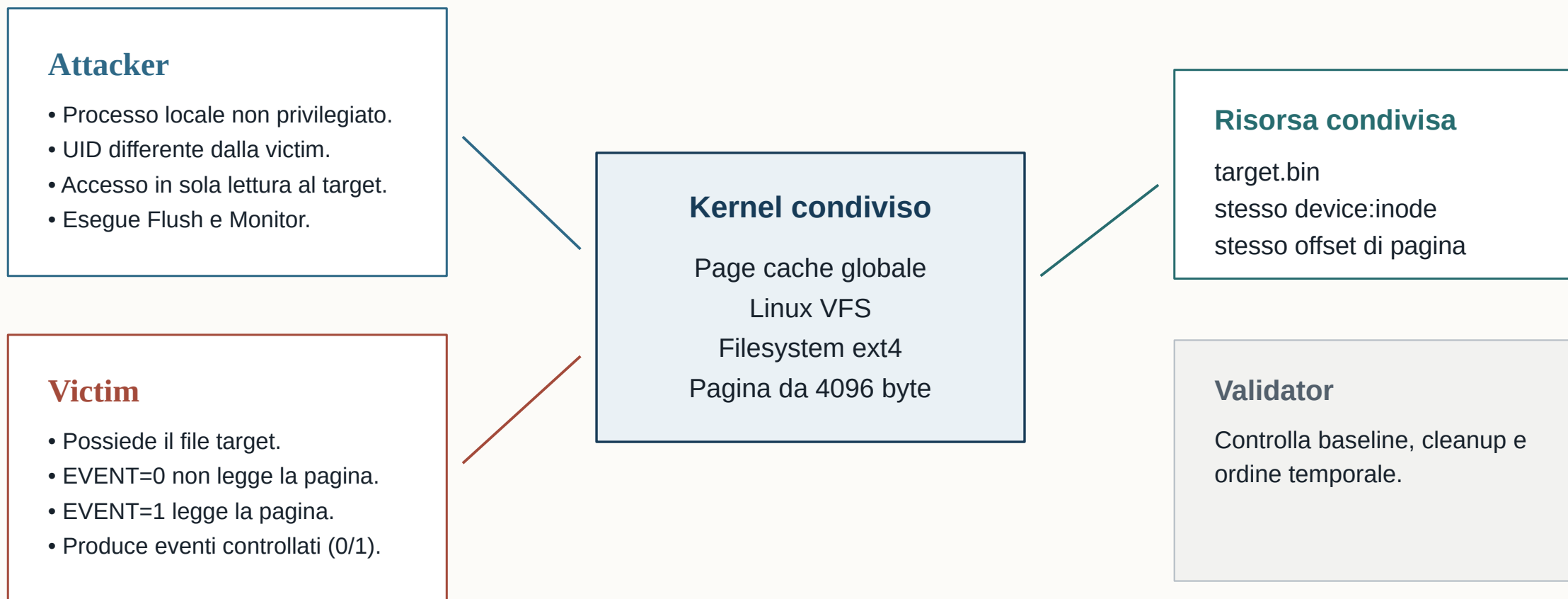
La proprietà “non contaminante” del Monitor è precisamente quella che deve essere riesaminata sui kernel moderni.

SEZIONE 2

Threat model, mitigazioni storiche e paper

Definiamo con precisione chi attacca chi, quali risorse sono condivise e quale informazione viene inferita. Solo a questo punto possiamo confrontare le primitive disponibili all'epoca del paper con quelle utilizzabili oggi.

Threat model del nostro esperimento: attori e risorse



Che cosa può e non può fare l'attacker

Capacità ammesse	Capacità escluse
Eseguire codice non privilegiato sulla stessa macchina.	Leggere la memoria privata o i registri della victim.
Aprire in lettura lo stesso file e conoscere la pagina target.	Usare ptrace, privilegi root o capacità speciali.
Invocare posix_fadvise e preadv2 sul proprio file descriptor.	Osservare pagine che non condividono lo stesso inode.
Misurare il tempo e ripetere il ciclo su più finestre.	Modificare il file target o il comportamento della victim.
Condividere kernel e page cache, anche da un container distinto.	Attaccare da remoto o ottenere esecuzione arbitraria nella victim.

Obiettivo di sicurezza. Distinguere se la victim ha eseguito l'accesso alla pagina target durante la finestra W.

L'output dell'attacker è un bit di attività. Per attribuirgli un significato applicativo occorre conoscere in anticipo la relazione fra quella pagina e un evento della victim.

Non stiamo dimostrando un exploit di code execution: stiamo dimostrando una violazione di confidenzialità tramite un side channel locale.

Dal bit di cache a un evento applicativo

Una pagina diventa utile quando contiene dati o codice consultati in corrispondenza dell'evento che si vuole rilevare. Il side channel osserva l'accesso, non il valore letto.

- 1** Analisi preliminare: si identifica un file condiviso e un offset correlato all'evento.
- 2** L'attacker rimuove quella pagina prima dell'intervallo di interesse.
- 3** Se la victim percorre il code path o legge il dato, la pagina torna residente.
- 4** Il Monitor trasforma la residency finale in una predizione binaria.

Esempio controllato usato nel progetto

Event o	Codice victim	Cache finale	Predizione
0	nessun pread	miss	0
1	pread page 0	hit	1

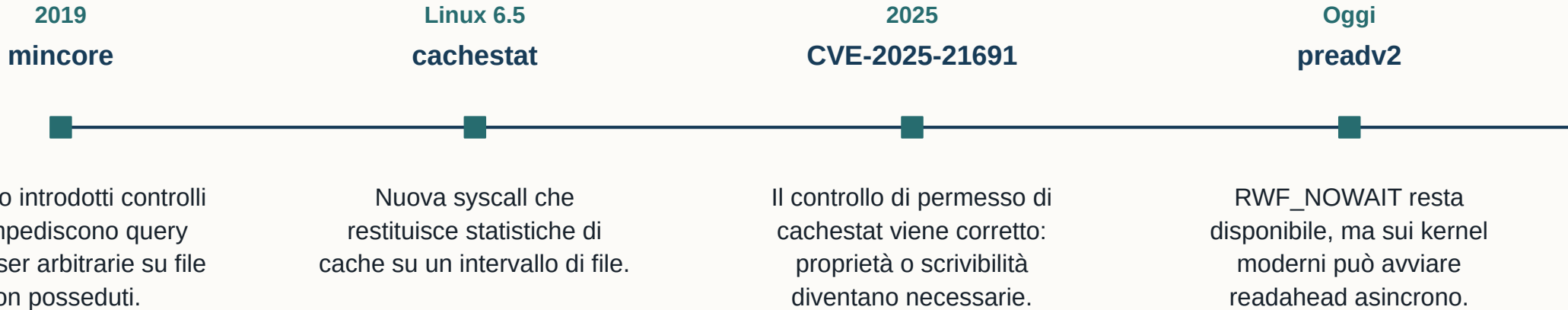
Il target è volutamente una sola pagina: elimina ambiguità e permette di misurare separatamente il meccanismo.

Possibile rumore: la stessa pagina può contenere più funzioni o essere letta da altri processi. In un attacco reale questo produce falsi positivi e richiede profilazione.

La demo Zenity applica questa idea alla pagina iniziale dell'eseguibile: l'avvio della UI diventa il segnale temporale.

Mitigazioni storiche: si restringe l'oracolo, non la condivisione

Linux ha più volte limitato le syscall che espongono la residency. La page cache, tuttavia, resta condivisa perché è un'ottimizzazione fondamentale del sistema.



Interpretazione

Le correzioni per-syscall rimuovono una specifica interfaccia di osservazione. Finché un processo può influenzare e interrogare lo stato della stessa pagina con altre primitive, il canale può riapparire in forma diversa.

Questo spiega perché il progetto non tenta di riabilitare cachestat: usa l'alternativa cross-user basata su preadv2 e ne studia la semantica corrente.

Che cosa mostra Eviction Notice

Il contributo del paper non è una sola syscall: è una sistematizzazione delle primitive di rimozione e osservazione della page cache e delle loro composizioni.

- Definisce Flush, Evict, Monitor e Reload come blocchi riutilizzabili.
- Costruisce cinque tecniche generiche, fra cui Flush+Monitor.
- Valuta attacchi locali e un covert channel su sistemi Linux reali.
- Mostra che restringere mincore non elimina il problema di fondo.

Ambiente e oracoli rilevanti

Aspetto	Paper
Kernel principale	Linux 6.8
Verifica aggiuntiva	Linux 6.12.4
Monitor più rapido	cachestat
Alternativa cross-user	preadv2 + RWF_NOWAIT
Risoluzione spaziale	una pagina, tipicamente 4 KiB

Il nostro punto di partenza

Riproduciamo l’artifact e poi ci concentriamo sul percorso preadv2, perché la mitigazione di cachestat impedisce il threat model cross-user che ci interessa.

La compatibilità dichiarata dal paper e il nostro problema di post-stato non sono in contraddizione: osservano proprietà diverse del ciclo, come vedremo nella sezione successiva.

Il Monitor basato su preadv2 nel paper

RWF_NOWAIT chiede che la read non attenda l'I/O. Il primo valore di ritorno viene usato come oracolo di residency.

```
flush(fd, page);  
wait_observation_window(W);  
r = preadv2(fd, &byte, 1, 0, RWF_NOWAIT);  
  
if (r == 1)  
    prediction = HIT;  
else if (r == -1 && errno == EAGAIN)  
    prediction = MISS;
```

Ritorno 1

La pagina era disponibile senza attendere: viene classificata come residente, quindi EVENT=1.

Ritorno -EAGAIN

La pagina non era immediatamente disponibile: viene classificata come assente, quindi EVENT=0.

Proprietà desiderata del Monitor: distinguere hit e miss senza trasformare il miss in un futuro hit causato dall'attacker stesso.

Sul kernel moderno la prima classificazione resta utilizzabile, ma la chiamata può avviare readahead dopo aver restituito EAGAIN. È il post-stato, non il bit iniziale, a richiedere un adattamento.

Quali numeri del paper sono confrontabili — e quali no

Risultato del paper	Valore	Che cosa misura
Risoluzione spaziale	4 KiB	Granularità della pagina osservata.
Limite temporale della primitiva	≈ 0,8 μs	Costo/upper bound della primitiva nel sistema del paper, prevalentemente con cachestat.
Covert channel	37,7 kB/s	Throughput applicativo ottenuto nella configurazione del paper.
Kernel	6.8; verifica 6.12.4	Portabilità delle tecniche valutate dagli autori.

Il paper non sceglie una singola “finestra W” equivalente al nostro sweep

La nostra finestra parte dal deadline della victim e termina al Monitor: include scheduling, esecuzione della victim e I/O. Il valore ≈0,8 μs del paper descrive invece la primitiva/oracolo. Mettere i due numeri nella stessa colonna come se fossero velocità alternative sarebbe metodologicamente scorretto.

Il confronto corretto è qualitativo: il paper dimostra la capacità dell’attacco; noi verifichiamo la ripetibilità cross-user del percorso preadv2 e troviamo la minima W affidabile nel nostro protocollo.

SEZIONE 3

Adattare il Monitor a Linux 7.2

Riproduciamo prima il comportamento di riferimento su Linux 5.15. Poi isoliamo il cambiamento introdotto dal readahead con IOCB_NOWAIT e costruiamo un ciclo che ristabilisce un post-stato noto prima del round successivo.

Ambienti usati e ruolo di ciascuno

Ambiente	Kernel / filesystem	Scopo	Vincoli
VM reference	5.15.0-187 · ext4	Riproduzione della semantica originale	Ubuntu Jammy; 2 vCPU
VM modern	7.2.0-flushmon · ext4	Adattamento, sweep e mitigazione	Ubuntu Jammy; 2 vCPU
Host	kernel host · Btrfs	Solo demo locale e cross-container	RWF_DONTCACHE non supportato da Btrfs

Perché due VM fisse

- Il kernel e il filesystem sono parte dell’oggetto di studio: mantenerli fissi evita che un aggiornamento del sistema alteri la semantica durante la campagna.
- Le due vCPU consentono di separare attacker e victim; le scadenze assolute monotone riducono la deriva temporale fra i round.
- La pagina target è identica in tutti gli esperimenti: file di 4096 byte, stesso inode condiviso fra due utenti differenti.

Le VM non servono a simulare un attacco remoto. Servono a controllare il kernel e a rendere riproducibile il confronto fra semantica precedente e moderna.

Riproduzione preliminare del meccanismo

Prima della campagna temporale abbiamo verificato separatamente Flush e condivisione cross-user.

Verifica	Stato iniziale	Azione	Stato osservato	Conclusione
Read / Flush	file non residente	lettura di 64 pagine	64 residenti; 0 dopo Flush	Il Flush ristabilisce C=0.
Cross-user	C=0 impostato dall'attacker	secondo UID legge il file	le pagine tornano residenti	La cache è condivisa fra utenti.

Risultato

Il Flush produce un baseline noto; l'accesso del secondo UID rende nuovamente residente la stessa pagina.

Passi successivi: adattare il Monitor al kernel moderno e misurare la finestra end-to-end.

Il cambiamento moderno: NOWAIT può avviare readahead

Il commit rimuove l'implicazione automatica `RWF_NOWAIT` → `IOCB_NOIO`. La syscall non attende l'I/O, ma può programmarlo prima di restituire `EAGAIN`.

Modifica essenziale

```
- kiocb_flags |= IOCB_NOIO;  
+ page_cache_sync_readahead(...);
```

7 linee modificate nel commit



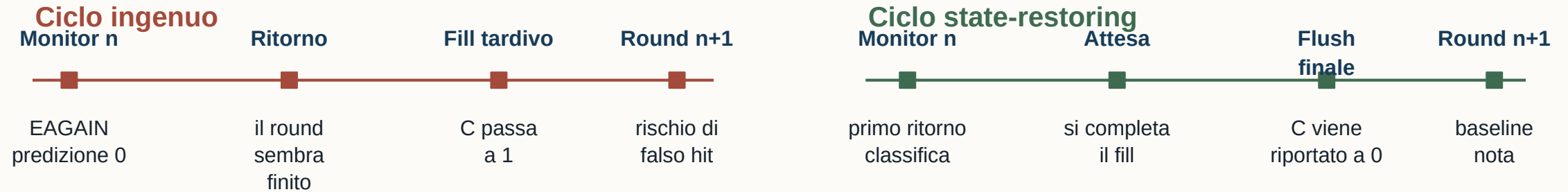
Distinzione fondamentale

`EAGAIN` descrive correttamente lo stato al momento del primo probe: la pagina non era disponibile. Tuttavia, dopo il ritorno la pagina può comparire per effetto dello stesso probe. La chiamata è non bloccante, non necessariamente non contaminante.

La traccia locale su Linux 7.2 osserva esattamente la sequenza `EAGAIN` → `fill asincrono`; sulla reference 5.15 il miss rimane assente.

Una predizione corretta può lasciare uno stato sbagliato

Per un esperimento ripetibile non basta produrre il bit corretto: ogni round deve terminare con la stessa postcondizione con cui inizierà il round successivo.



Invariante richiesto dal progetto

Dopo ogni Monitor e prima di accettare il round come concluso deve valere:

$$C(\text{target}, \text{page } 0, \text{cleanup_end}) = 0$$

Il validator misura questa postcondizione al termine di ogni trial.

Confronto con la verifica del paper su 6.12.4

Aspetto	Eviction Notice	Questo progetto
Domanda principale	Le tecniche page-cache sono ancora utilizzabili?	Il ciclo preadv2 cross-user è ripetibile con post-stato noto?
Oracolo più rapido	Prevalentemente cachestat	preadv2 + RWF_NOWAIT
Proprietà comune	Il primo ritorno distingue hit e miss	Il primo ritorno distingue hit e miss
Proprietà aggiuntiva	Compatibilità generale su 6.12.4	Controllo del fill asincrono e cleanup su 7.2
Scala temporale	Primitive e attacchi applicativi	10.000 round per finestra ravvicinata

Differenza

Il paper verifica la disponibilità delle tecniche; questo progetto misura la ripetibilità del ciclo preadv2 e richiede che ogni round ripristini la pagina.

L'adattamento riguarda il post-stato del ciclo preadv2 sui kernel moderni.

Variante A: attendere il fill e applicare Flush

```
1 flush(target_page)
2 wait(W)
3 r = probe(RWF_NOWAIT)
4 prediction = (r == 1)
5 while r == -EAGAIN:
6 r = probe(RWF_NOWAIT)
7 flush(target_page)
8 return prediction
```

Perché funziona

- 1 La predizione usa esclusivamente il primo ritorno, prima che il readahead completi.
- 2 Sul miss si aspetta che l'I/O avviato dal kernel abbia raggiunto uno stato stabile.
- 3 Il Flush successivo rimuove la pagina; non può essere scavalcato da un fill ancora pendente.

Costo

Il miss diventa più lento perché il cleanup deve attendere un I/O che il Monitor originale non attendeva.

La soluzione privilegia una postcondizione esplicita e usa primitive disponibili anche sul filesystem Btrfs del sistema host.

Variante B: aggiungere RWF_DONTCACHE

RWF_DONTCACHE chiede al kernel di ridurre la permanenza in cache dei dati letti. Lo combiniamo con NOWAIT e manteniamo la classificazione basata sul primo ritorno.

```
flags = RWF_NOWAIT | RWF_DONTCACHE
r = preadv2(fd, &byte, 1, 0, flags)
prediction = (r == 1)

while r == -EAGAIN:
    r = probe(RWF_NOWAIT)
    flush(target_page)
```

Nel worker formale

- Il primo probe usa RWF_NOWAIT | RWF_DONTCACHE.
- Sul miss si completa il readahead pendente.
- Un Flush finale uniforma la postcondizione e permette al validator di controllarla nello stesso modo per entrambe le varianti.

Portabilità

ext4 accetta il flag e produce il miglior risultato dello sweep. Btrfs restituisce EOPNOTSUPP: per la demo sul host usiamo wait+Flush.

DONTCACHE non è una garanzia universale di assenza dalla cache; è un suggerimento di drop-behind. Per questo la campagna misura e valida comunque il cleanup.

Costo isolato e proprietà dei tre Monitor

Monitor	Kernel	Probe iniziale	miss p50	hit p50	Ripristino
Reference	5.15	NOWAIT	0,232 µs	0,777 µs	Miss non contaminante
wait+Flush	7.0.3	NOWAIT	13,689 µs	0,814 µs	Attesa fill + DONTNEED
DONTCACHE	7.0.3	NOWAIT DONTCACHE	13,169 µs	0,820 µs	Drop-behind + normalizzazione

Interpretazione dei tempi

- Sull'hit i tre valori restano vicini: la pagina è già disponibile e non si attende il readahead.
- Sul miss le varianti moderne pagano circa 13 µs per completare e ripulire l'effetto asincrono.
- DONTCACHE è leggermente più rapido su ext4, ma il vantaggio è piccolo e non compensa la minore portabilità.
- Questi sono tempi del ciclo isolato, non la finestra end-to-end necessaria alla victim: le due metriche verranno tenute separate.

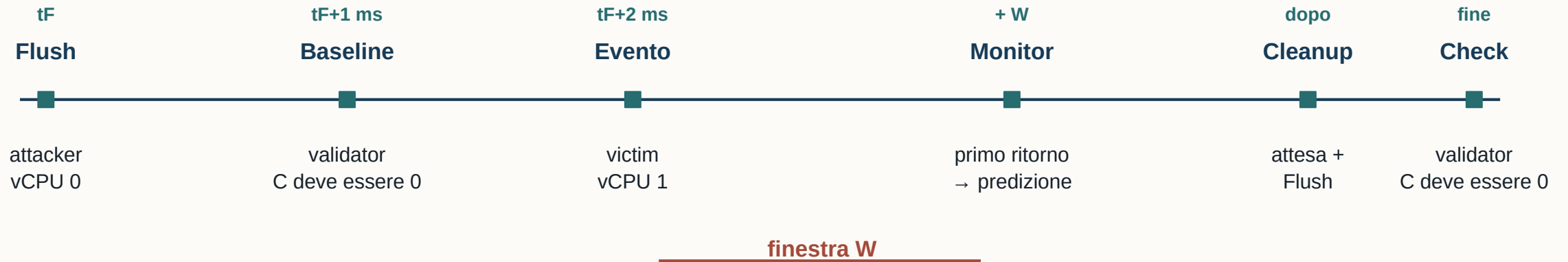
SEZIONE 4

Protocollo sperimentale e finestra minima

Dopo aver ottenuto un Monitor con post-stato controllato, misuriamo la domanda che interessa il progetto: quanto può essere corta la finestra lasciata alla victim mantenendo un risultato affidabile e statisticamente supportato?

Protocollo di un trial nello sweep

Attacker, victim e validator sono processi persistenti. Ogni trial usa scadenze assolute `CLOCK_MONOTONIC` per evitare che il ritardo si accumuli.



Separazione dei dati

Lo script registra EVENT, prediction e tempi. L'analisi unisce i dati per calcolare le metriche.

Controllo del rumore

Attacker e victim sono pinning su vCPU diverse. La stessa sequenza pseudocasuale di eventi viene riusata per confrontare le varianti senza cambiare la composizione 0/1.

Come definiamo la finestra minima affidabile

Definizione. $W = \text{monitor_deadline} - \text{victim_event_deadline}$. Una W più piccola offre migliore risoluzione temporale, ma aumenta la probabilità che $\text{EVENT}=1$ non abbia completato la read.

Criteri di accettazione · 10.000 trial

- $\text{accuracy} \geq 99\%$
- $\text{recall} \geq 99\%$
- $\text{false-positive rate} \leq 1\%$
- $\text{cleanup failure rate} \leq 1\%$
- $\text{timing inversion rate} \leq 1\%$

Perché usiamo i bound di Wilson

Una recall osservata appena sopra 99% non basta: la sua incertezza potrebbe includere valori inferiori alla soglia. Richiediamo quindi il limite inferiore al 95% $\geq 99\%$; per errori e FPR usiamo il limite superiore $\leq 1\%$.

In questo modo “confermato” ha un significato statistico esplicito.

Regola della finestra inferiore

Accettiamo una minima soltanto se la finestra immediatamente più corta è stata realmente eseguita e respinta. Evitiamo quindi di chiamare “minimo” il più piccolo valore provato per caso.

Baseline: correttezza prima dell’ottimizzazione temporale

La baseline verifica che le tre implementazioni classifichino lo stesso evento in una condizione temporale larga, prima di cercare la risoluzione minima

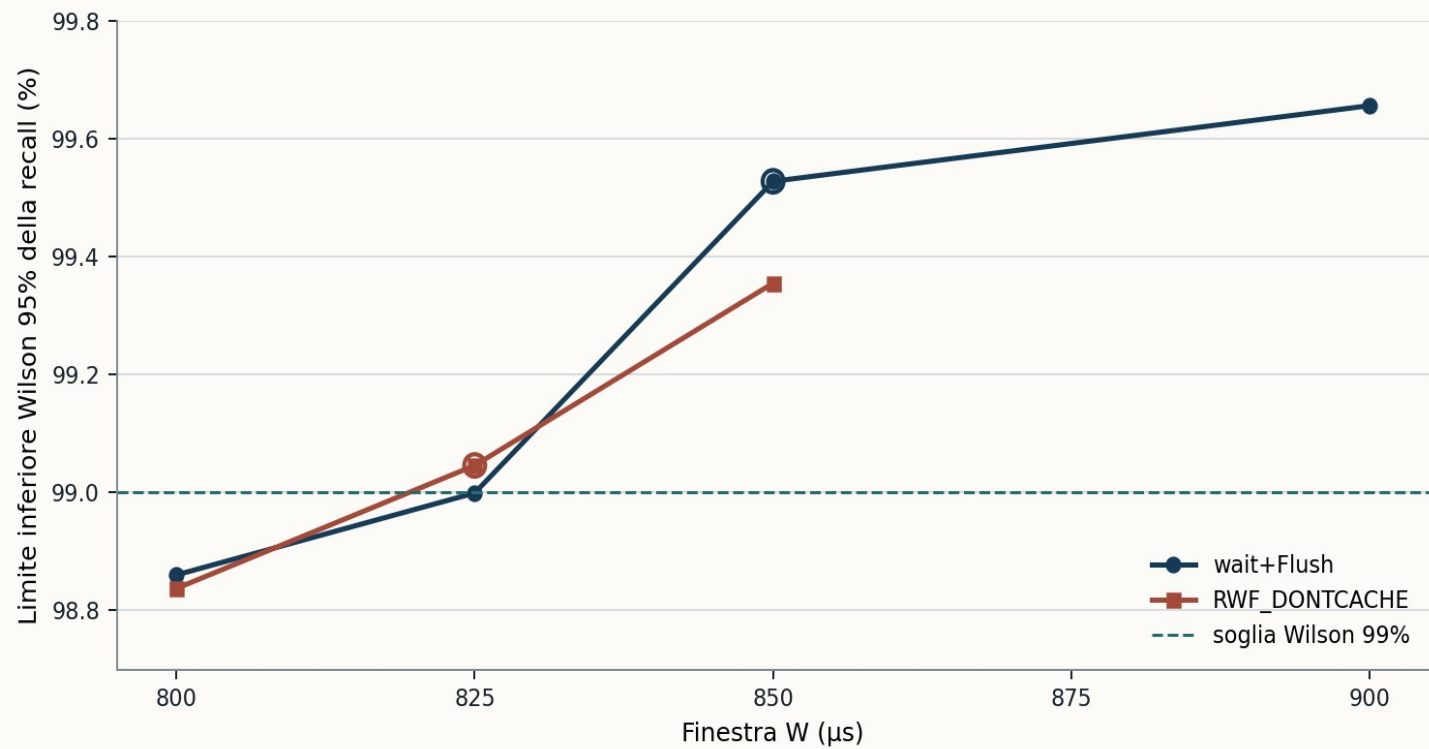
Monitor	Kernel	TP	TN	FP	FN	Accuracy	Recall
Reference upstream	5.15	103	97	0	0	100%	100%
wait+Flush	7.2	103	97	0	0	100%	100%
RWF_DONTCACHE	7.2	103	97	0	0	100%	100%

Cosa possiamo concludere

La semantica binaria è riprodotta in tutte le condizioni: hit corrisponde a EVENT=1 e miss a EVENT=0. Le varianti moderne non introducono errori quando W è molto più grande del tempo di esecuzione della victim.

Cosa non possiamo concludere: 200/200 a 50 ms non stabilisce la risoluzione temporale, né fornisce evidenza sufficiente vicino alla soglia del 99%. Da qui la campagna da 10.000 trial per finestra.

Sweep: prima finestra confermata per ciascun Monitor



Variante	W	Recall	Wilson low
DONTCACHE	825 μs	99,316%	99,046%
wait+Flush	850 μs	99,718%	99,528%

Finestre inferiori respinte

DONTCACHE 800 μs:
Wilson low recall 98,837%

wait+Flush 825 μs:
Wilson low recall 98,9993%

Il margine di DONTCACHE è 25 μs (circa 3%). Il risultato non è “zero errori”: è la prima W che soddisfa insieme soglie puntuali, intervalli di confidenza e controllo del cleanup.

Confronto metodologico con Eviction Notice

Dimensione	Eviction Notice	Questo progetto
Obiettivo	Dimostrare e generalizzare page-cache attacks	Rendere ripetibile il Monitor preadv2 su Linux 7.2
Oracle	cachestat nei risultati più rapidi; preadv2 alternativo	preadv2 cross-user con ciclo di ripristino
Misura temporale	≈0,8 μs per la primitiva	W end-to-end: 825 / 850 μs
Contenuto di W	Non definita come nel nostro sweep	Scheduling + read victim + Monitor
Evidenza	Attacchi e covert channel	10.000 trial per finestra + bound Wilson
Risultato aggiuntivo	Compatibilità fino a 6.12.4	Post-stato esplicito, cleanup validato e mitigazione misurata

Estensione sperimentale sviluppata nel progetto

Abbiamo identificato l'effetto del readahead sul post-stato, implementato due varianti compatibili con il primo ritorno di preadv2 e determinato sperimentalmente la finestra minima affidabile nel nostro ambiente moderno.

SEZIONE 5

Mitigazione, costo e dimostrazioni

Infine interveniamo sulla victim: invece di inseguire una nuova syscall da bloccare, eliminiamo la dipendenza fra evento e accesso osservabile. Valutiamo la perdita d'informazione, il costo e la persistenza del canale attraverso i container.

Mitigazione constant-access: rendere uguali i due percorsi

La mitigazione non cerca di tenere la pagina fuori dalla cache. Fa in modo che la pagina venga letta sia per EVENT=0 sia per EVENT=1, così la residency non dipende più dal bit sensibile.

Policy	EVENT=0	EVENT=1	Uso del byte	Residency osservabile
normal	nessuna read	pread pagina 0	solo EVENT=1	miss / hit → 1 bit
constant	dummy pread	pread pagina 0	solo EVENT=1	hit / hit → costante

Senza mitigazione

$$C_{\text{final}} = \text{EVENT}$$

La residency è una funzione diretta dell'evento e può trasportare un bit per trial.

Con constant-access

$$C_{\text{final}} = 1$$

La residency è costante; il byte letto per EVENT=0 viene semplicemente ignorato.

Limite: la tecnica protegge solo percorsi per i quali è possibile equalizzare gli accessi osservabili; non è una difesa generale per qualsiasi side channel di page cache.

Valutazione della sicurezza: da un bit a nessuna informazione

Victim	Monitor	Accuracy	TPR	FPR	TPR-FPR	Mutual information
normal	wait+Flush	100%	1	0	1	1 bit
normal	DONTCACHE	100%	1	0	1	1 bit
constant	wait+Flush	50%	1	1	0	0 bit
constant	DONTCACHE	50%	1	1	0	0 bit

Perché accuracy = 50% non è da sola una prova di sicurezza

Con constant-access l'attacker osserva sempre hit e predice sempre 1. Su eventi bilanciati questo produce 50% di accuracy, ma il risultato sostanziale è $TPR=FPR$: la distribuzione dell'osservazione è identica nei due casi. Per questo il vantaggio è 0 e la mutual information stimata è 0 bit.

La mitigazione non rende il Monitor meno preciso: rende irrilevante il bit che il Monitor osserva.

Costo della mitigazione sulla victim

Evento	Victim normal	Victim constant	Differenza media	Interpretazione
EVENT=0	50,42 ns	431,16 ns	+380,73 ns	Costo della dummy read
EVENT=1	436,54 ns	427,37 ns	-9,17 ns	Differenza entro il rumore
Media bilanciata	243,48 ns	429,26 ns	+185,78 ns	Costo medio per operazione

Lettura corretta della percentuale

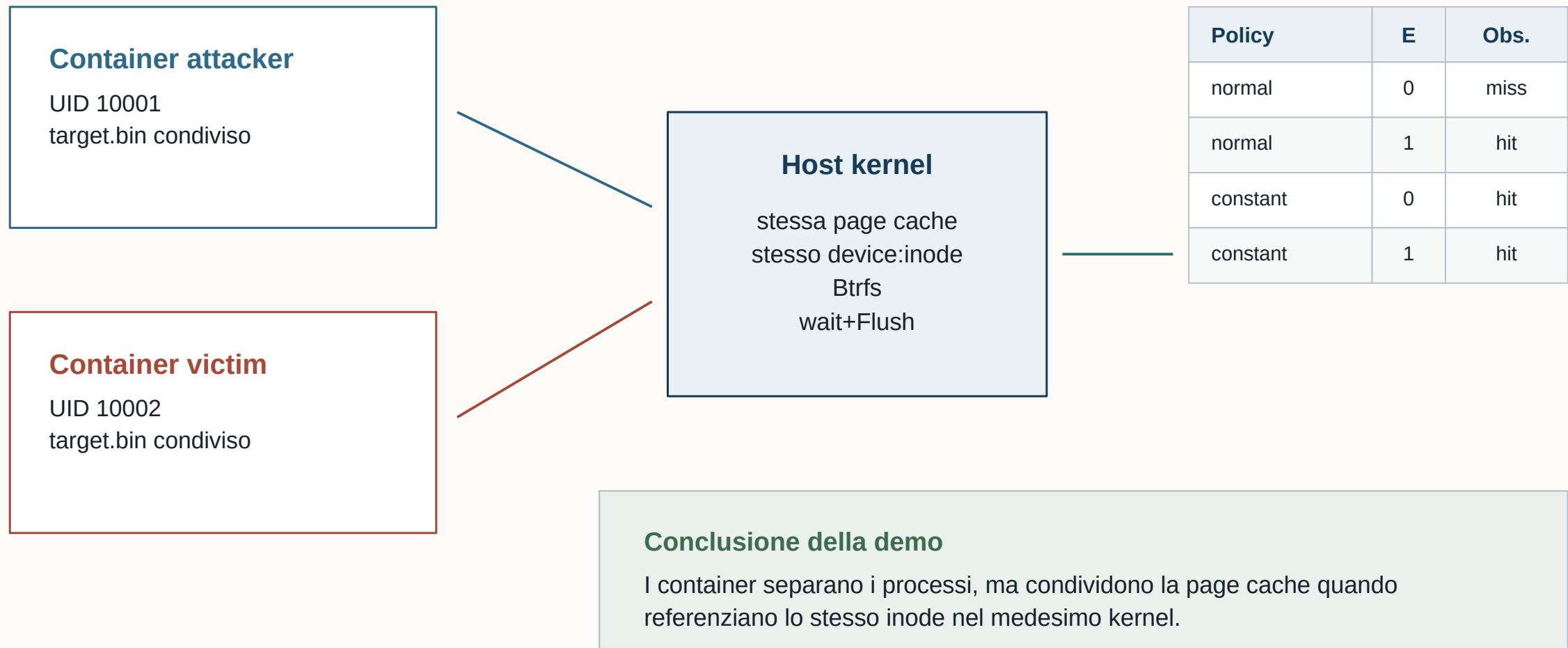
EVENT=0 cresce del 755%, ma il baseline è circa 50 ns.
La misura utile per ragionare sul costo è l'incremento assoluto di 380,73 ns, non la sola percentuale.

Validità esterna

Il benchmark usa una pagina già calda e isola una singola operazione. In un'applicazione reale il costo dipende da frequenza degli eventi, locality, filesystem e pressione di memoria.

La mitigazione ha quindi un costo misurabile ma non “distrugge le prestazioni” in modo universale: il giudizio dipende dal numero di accessi equalizzati nel programma reale.

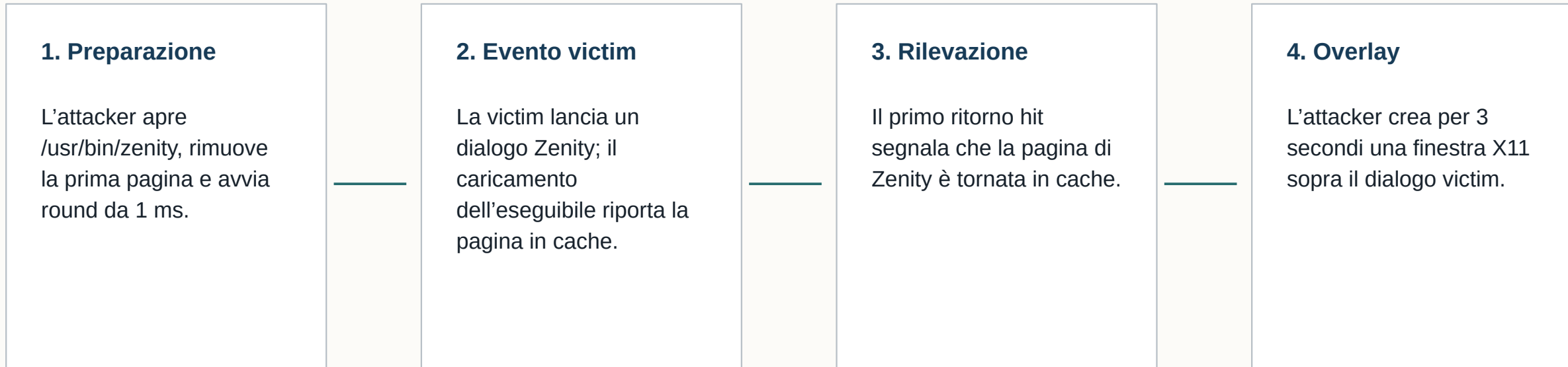
Demo cross-container: isolamento dei processi, non della page cache



Lo script confronta ogni evento noto con l'osservazione restituita dall'attacker.

Demo UI redressing: dal segnale all'azione temporale

La demo usa Flush+Monitor per rilevare l'avvio di un dialogo Zenity e sovrapporre una finestra controllata dall'attacker.



Risultato

Il segnale di cache attiva un overlay separato, senza eseguire nuovamente Zenity.

Conclusioni, contributi e limiti

Risultati ottenuti

- 1 Riproduzione del canale e della separazione cross-user sulla reference 5.15.
- 2 Analisi causale del fill asincrono prodotto da RWF_NOWAIT sul kernel 7.2.
- 3 Due varianti con post-stato controllato e compatibilità filesystem distinta.
- 4 Finestra minima confermata: 825 μ s DONTCACHE, 850 μ s wait+Flush.
- 5 Mitigazione constant-access: mutual information da 1 a 0 bit, +185,78 ns medi bilanciati.

Limiti dell'evidenza

- Esperimento controllato su una pagina e due VM specifiche.
- La finestra dipende da hardware, scheduler, kernel e filesystem.
- RWF_DONTCACHE è best effort e non supportato da Btrfs nel nostro host.
- Constant-access protegge solo accessi che possono essere equalizzati.
- La demo UI dipende dalla pagina Zenity scelta e da X11/XWayland per controllare lo stacking.

Risposta alla domanda iniziale: sì, Flush+Monitor resta dimostrabile su un kernel moderno nel threat model cross-user, purché il Monitor consideri il primo ritorno come osservazione e ristabilisca esplicitamente lo stato prima del round successivo.

Dati, codice e demo sono contenuti nel repository flush-monitor.

Riferimenti principali e tracciabilità dei risultati

Fonte	Uso nella presentazione
Eviction Notice: Reviving and Advancing Page Cache Attacks, NDSS 2026, doi:10.14722/ndss.2026.240006	Modello delle primitive, tecniche, risultati e ambiente del paper.
Linux commit 0a2d82946be6, mm: allow read-ahead with IOCB_NOWAIT set	Causa del fill asincrono successivo al primo EAGAIN.
CVE-2025-21691, cachestat permission checking	Mitigazione dell'oracolo cachestat nel threat model cross-user.
Linux man-pages, preadv2(2)	Semantica di RWF_NOWAIT e RWF_DONTCACHE.
artifact/ · commit 84c57881855e	Codice originale riprodotto sulla VM reference.
data/processed/baseline_metrics.csv	Baseline 200/200 sulle tre condizioni.
data/processed/window-sweep-minima.csv data/processed/window-sweep-metrics.csv	Sweep da 10.000 trial e finestre minime confermate.
data/processed/mitigation-security-metrics.csv data/processed/mitigation-victim-overhead.csv	Perdita d'informazione e costo della mitigazione.

Il repository conserva sia i dati grezzi sia le tabelle elaborate: ogni valore presentato può essere ricondotto alla relativa campagna.